Technical Specification Document (TSD)

Project Name

GigSave – Smart Expense & Goal Tracker for Gig Workers

Version: 1.0
Technology Stack: React.js + Supabase + JavaScript + Vercel

---

1. Introduction

1.1 Purpose

This Technical Specification Document (TSD) defines the complete technical architecture, technology stack, database design, APIs, application modules, security mechanisms, deployment strategy, and development standards for the GigSave application.

---

2. System Overview

GigSave is a cloud-based web application that enables gig workers to:

- Record daily income
- Record expenses
- Automatically allocate savings into jars
- Track financial goals
- Analyze spending patterns
- Receive AI-powered financial insights
- Access their data securely from anywhere

---

3. Technology Stack

Frontend

Technology	Purpose
React.js	User Interface
JavaScript (ES6+)	Application Logic
React Router	Navigation
Tailwind CSS	UI Styling
React Hook Form	Form Handling
Recharts	Analytics Charts
Axios	API Communication
Lucide React	Icons

Backend

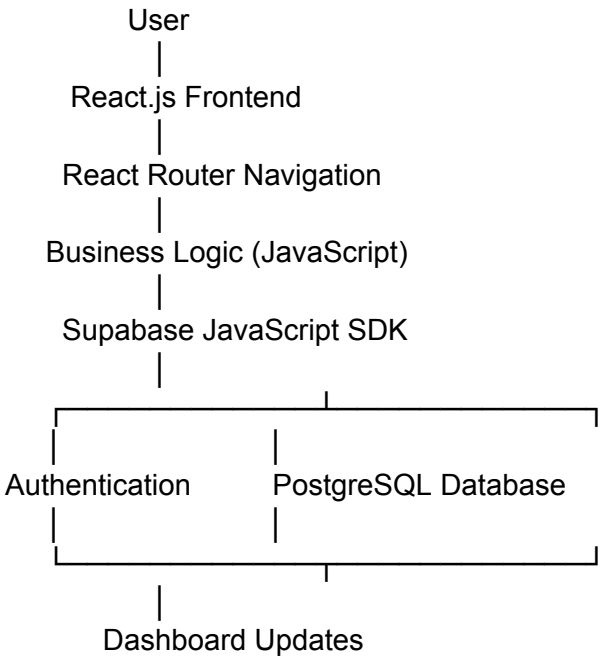
Technology	Purpose
Supabase	Backend as a Service
PostgreSQL	Database
Supabase Auth	Authentication
Supabase Storage	Profile Images
Row Level Security	Data Protection

Hosting

Service	Purpose
Vercel	Frontend Deployment
Supabase Cloud	Backend & Database

---

4. System Architecture



---

5. Application Modules

Module 1 – Authentication

Features

- User Registration
- Login
- Logout
- Forgot Password
- Session Management

Supabase Services

- signUp()
- signInWithPassword()
- signOut()
- resetPasswordForEmail()

Module 2 – Dashboard

Displays:

- Today's Income
- Today's Expenses
- Total Savings
- Available Balance
- Goal Progress
- Financial Health Score

Data Sources

- income
- expenses
- jars
- goals

Module 3 – Income Management

Functions

- Add Income
- Edit Income
- Delete Income
- View Income History

Fields

- Amount
- Source
- Date
- Notes

Module 4 – Expense Management

Functions

- Add Expense
- Update Expense
- Delete Expense

Categories

- Fuel
- Food
- Rent
- Shopping

- Medical
- Recharge
- Other

Module 5 – Savings Jars

Each user can create multiple jars.

Example

Emergency Fund

Vehicle Maintenance

Family Support

Investment

Education

Vacation

Each jar contains:

- Name
- Percentage
- Balance
- Color/Icon

Module 6 – Goals

Each goal contains:

- Name
- Target Amount
- Linked Jar
- Current Progress
- Completion Percentage

Module 7 – Analytics

Charts

- Daily Income
- Weekly Income
- Monthly Income
- Expense Categories
- Savings Growth
- Goal Progress

Module 8 – AI Insights

Examples

- Spending Suggestions
- Saving Recommendations
- Goal Prediction
- Weekly Summary

Version 1 may use rule-based JavaScript logic. Future versions can integrate OpenAI or Groq APIs.

---

6. Database Design

Table: profiles

Column	Type
id	UUID (PK)
full_name	TEXT
phone	TEXT
occupation	TEXT
preferred_language	TEXT
created_at	TIMESTAMP

Table: income

Column	Type

id	UUID
user_id	UUID
amount	DECIMAL
source	TEXT
notes	TEXT
income_date	DATE
created_at	TIMESTAMP

Table: expenses

Column	Type
id	UUID
user_id	UUID
category	TEXT
amount	DECIMAL
note	TEXT
expense_date	DATE
created_at	TIMESTAMP

Table: jars

Column	Type
id	UUID
user_id	UUID
jar_name	TEXT
percentage	INTEGER
balance	DECIMAL

color	TEXT

Table: goals

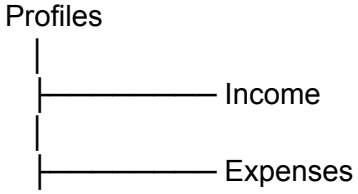
Column	Type
id	UUID
user_id	UUID
jar_id	UUID
goal_name	TEXT
target_amount	DECIMAL
current_amount	DECIMAL
deadline	DATE

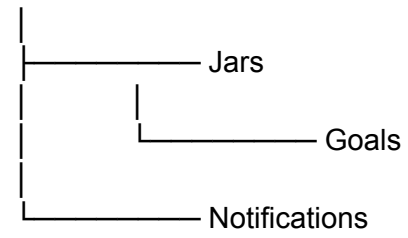
Table: notifications

Column	Type
id	UUID
user_id	UUID
title	TEXT
message	TEXT
is_read	BOOLEAN
created_at	TIMESTAMP

---

7. Database Relationships





---

8. Business Logic

Income Allocation Algorithm

Input Daily Income

↓

Read User Jar Percentages

↓

Calculate Amount for Every Jar

↓

Update Jar Balances

↓

Remaining Amount

↓

Available Balance

Example

Income = ₹1000

Emergency (20%) = ₹200

Bike = ₹100

Investment = ₹150

Available = ₹550

Financial Health Score

Formula

Savings Score

+

Goal Completion

+

Expense Control

+

Recording Consistency

=

Health Score (0–100)

Goal Progress

(Current Amount ÷ Target Amount)

×

100

---

9. Folder Structure

src/

├── assets/

├── components/

│ ├── Navbar.jsx
│ ├── Sidebar.jsx

```
│   ├── IncomeCard.jsx
│   ├── ExpenseCard.jsx
│   ├── GoalCard.jsx
│   ├── JarCard.jsx
│   └── Charts/

├── pages/

│   ├── Login.jsx
│   ├── Register.jsx
│   ├── Dashboard.jsx
│   ├── Income.jsx
│   ├── Expenses.jsx
│   ├── Jars.jsx
│   ├── Goals.jsx
│   ├── Analytics.jsx
│   └── Profile.jsx

├── services/

│   ├── authService.js
│   ├── incomeService.js
│   ├── expenseService.js
│   ├── jarService.js
│   ├── goalService.js

├── hooks/

├── context/

├── utils/

├── supabase/

│   └── client.js

├── routes/

├── App.jsx

└── main.jsx
```

---

10. API / Service Layer

Authentication

- Register User
- Login User
- Logout User

Income

- Create Income
- Update Income
- Delete Income
- Get User Income

Expenses

- Create Expense
- Update Expense
- Delete Expense
- Get Expenses

Jars

- Create Jar
- Update Jar
- Delete Jar
- Allocate Savings

Goals

- Create Goal
- Update Goal
- Delete Goal
- Get Goals

---

11. Security

Authentication

- Supabase Authentication

Authorization

- Row Level Security (RLS)

Password

- Managed securely by Supabase Auth

Communication

- HTTPS

Validation

- Client-side validation
- Server-side validation

---

12. Performance Requirements

Dashboard Load

≤ 2 seconds

Database Query

≤ 500 ms

Authentication

≤ 2 seconds

Analytics Rendering

≤ 3 seconds

---

13. Error Handling

Scenario	Action
Invalid login	Show authentication error
Network unavailable	Display offline message
Database failure	Retry request
Invalid input	Show validation message
Session expired	Redirect to login

---

14. Deployment

Frontend

- Build using Vite
- Deploy to Vercel

Backend

- Hosted on Supabase Cloud

Environment Variables

VITE_SUPABASE_URL=

VITE_SUPABASE_ANON_KEY=

---

15. Browser Compatibility

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari

Responsive support:

- Desktop
- Tablet
- Mobile

---

16. Future Technical Enhancements

- Progressive Web App (PWA)
- Voice recognition using Web Speech API
- AI integration using OpenAI/Groq
- Offline data synchronization
- SMS income detection
- UPI transaction import
- Push notifications
- Multi-language support
- Dark mode
- OCR for receipt scanning

---

17. Testing Strategy

Unit Testing

- Utility functions
- Business logic
- Savings allocation algorithm

Integration Testing

- React ↔ Supabase communication
- Authentication flow
- CRUD operations

User Acceptance Testing (UAT)

- Income tracking
- Expense tracking
- Jar allocation
- Goal management
- Dashboard analytics
- Authentication

---

18. Development Standards

- Follow ES6+ JavaScript standards.
- Use functional React components and Hooks.
- Maintain reusable components.
- Keep business logic in the service layer.
- Protect all database tables using Supabase RLS.
- Use environment variables for sensitive configuration.
- Write modular, maintainable, and documented code.

---

19. Conclusion

GigSave will be developed as a modern, cloud-based financial management platform using **React.js**, **Supabase**, and **JavaScript**. The architecture emphasizes simplicity, scalability, security, and maintainability while supporting future enhancements such as AI-powered financial coaching, voice input, and offline capabilities.